VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Shaikh Uzair Ahmed (1BM23CS307)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February-May 2025

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by Shaikh Uzair Ahmed **(1BM23CS307)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2024-25. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Dr. Sarala D V
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph. LeetCode Program related to Topological sorting	5
2	Implement Johnson Trotter algorithm to generate permutations.	10
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort. LeetCode Program related to sorting.	12
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken. LeetCode Program related to sorting.	15
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	19
6	Implement 0/1 Knapsack problem using dynamic programming. LeetCode Program related to Knapsack problem or Dynamic Programming.	22
7	Implement All Pair Shortest paths problem using Floyd's algorithm. LeetCode Program related to shortest distance calculation.	24
8	Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	27
9	Implement Fractional Knapsack using Greedy technique. LeetCode Program related to Greedy Technique algorithms.	34
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	36
11	Implement "N-Queens Problem" using Backtracking.	39

Course outcomes:

CO1	Analyze time complexity of Recursive and Non-recursive algorithms using asymptotic notations.
CO2	Apply various design techniques for the given problem.
CO3	Apply the knowledge of complexity classes P, NP, and NP-Complete and prove certain problems are NP-Complete
CO4	Design efficient algorithms and conduct practical experiments to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>

int a[10][10], n, t[10], indegree[10];
int stack[10], top = -1;

void computeIndegree(int, int [][10]);
void tps_SourceRemoval(int, int [][10]);

int main() {
    printf("Enter the no. of nodes: ");
    scanf("%d", &n);
    int i, j;

    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    computeIndegree(n, a);
    tps_SourceRemoval(n, a);
    printf("Solution: ");

    for (i = 0; i < n; i++) {
        printf("%d ", t[i]);
    }

    return 0;
}
```

```
void computeIndegree(int n, int a[][10]) {
    int i, j, sum = 0;

    for (i = 0; i < n; i++) {
        sum = 0;
        for (j = 0; j < n; j++) {
            sum = sum + a[j][i];
        }
        indegree[i] = sum;
    }
}

void tps_SourceRemoval(int n, int a[][10]) {
    int i, j, v;

    for (i = 0; i < n; i++) {
        if (indegree[i] == 0) {
            stack[++top] = i;
        }
    }

    int k = 0;
    while (top != -1) {
        v = stack[top--];
        t[k++] = v;

        for (i = 0; i < n; i++) {
            if (a[v][i] != 0) {
                indegree[i] = indegree[i] - 1;
                if (indegree[i] == 0) {
                    stack[++top] = i;
                }
```

```
        }
    }
  }
}
```

```
Enter the no. of nodes: 6
0 0 1 1 0 0
0 0 0 1 1 0
0 0 0 1 0 1
0 0 0 0 0 1
0 0 0 0 0 1
0 0 0 0 0 0
Solution: 1 4 0 2 3 5
```

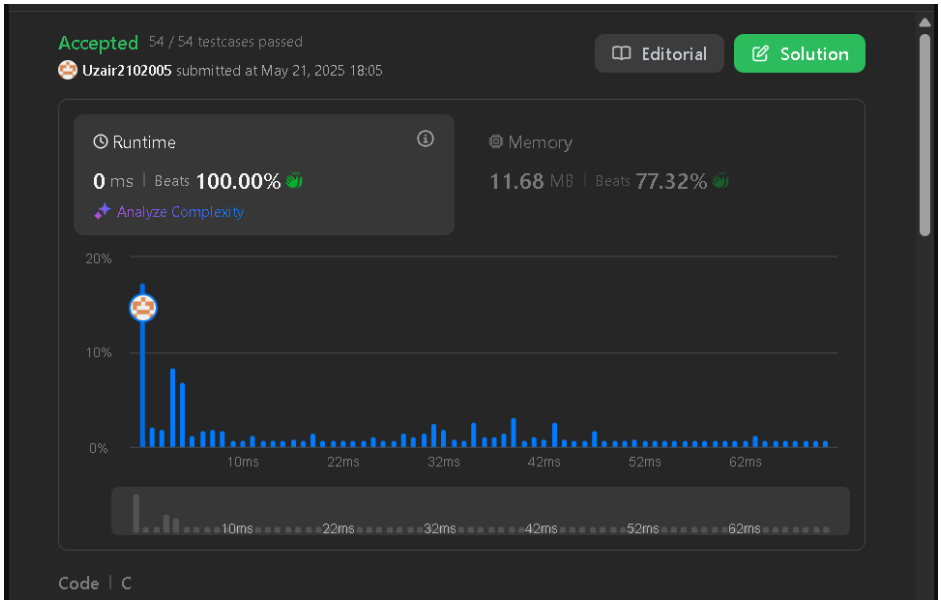
Output

LeetCode 207: Course Schedule

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize) {

    int* indegree = (int*)calloc(numCourses, sizeof(int));

    int** graph = (int**)calloc(numCourses, sizeof(int*));

    int* graphColSize = (int*)calloc(numCourses, sizeof(int));


    // First pass to count edges to allocate memory

    for (int i = 0; i < prerequisitesSize; i++) {

        int to = prerequisites[i][0];

        graphColSize[prerequisites[i][1]]++;

    }


    // Allocate space for adjacency list

    for (int i = 0; i < numCourses; i++) {

        graph[i] = (int*)malloc(graphColSize[i] * sizeof(int));

        graphColSize[i] = 0; // Reset to use it as an index for inserting

    }
```

```
    // Build graph and indegree array
    for (int i = 0; i < prerequisitesSize; i++) {
        int to = prerequisites[i][0];
        int from = prerequisites[i][1];
        graph[from][graphColSize[from]++] = to;
        indegree[to]++;
    }


    // Initialize queue
    int* queue = (int*)malloc(numCourses * sizeof(int));
    int front = 0, rear = 0;


    // Add courses with 0 indegree
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) {
            queue[rear++] = i;
        }
    }


    int count = 0;


    while (front < rear) {
        int course = queue[front++];
        count++;
        for (int i = 0; i < graphColSize[course]; i++) {
            int neighbor = graph[course][i];
            if (--indegree[neighbor] == 0) {
```

```
                queue[rear++] = neighbor;
            }
        }
    }


    // Cleanup
    for (int i = 0; i < numCourses; i++) {
        free(graph[i]);
    }
    free(graph);
    free(graphColSize);
    free(indegree);
    free(queue);


    return count == numCourses;
}
```



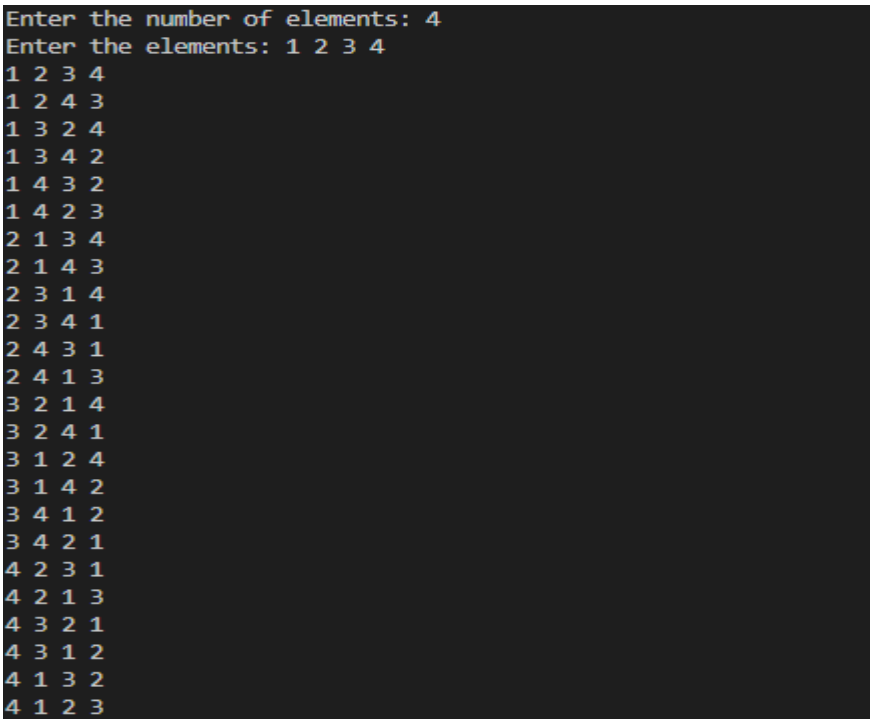
Output

Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>
#include <stdlib.h>

void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void generatePermutations(int arr[], int start, int end) {
    if (start == end) {
        for (int i = 0; i <= end; i++) {
            printf("%d ", arr[i]);
        }
        printf("\n");
    } else {
        for (int i = start; i <= end; i++) {
            swap(&arr[start], &arr[i]);
            generatePermutations(arr, start + 1, end);
            swap(&arr[start], &arr[i]); // backtrack
        }
    }
}

int main() {
    int n;
    printf("Enter the number of elements: ");
```

```
    scanf("%d", &n);

    int* arr = (int*)malloc(n * sizeof(int));
    printf("Enter the elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    generatePermutations(arr, 0, n - 1);
    free(arr);

    return 0;
}
```



Output

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>


int *a;


void mergeSort(int a[], int low, int high);

void merge(int a[], int low, int high, int mid);


int main() {

   int n;

   printf("Enter Number of Elements");

   scanf("%d", &n);

   a = (int*)malloc(sizeof(int) * n);



   for (int i = 0; i < n; ++i) {

      a[i]=rand();

   }



   clock_t start = clock();

   mergeSort(a, 0, n - 1);

   clock_t end = clock();



   double t = ((double)(end - start)) / CLOCKS_PER_SEC;
```

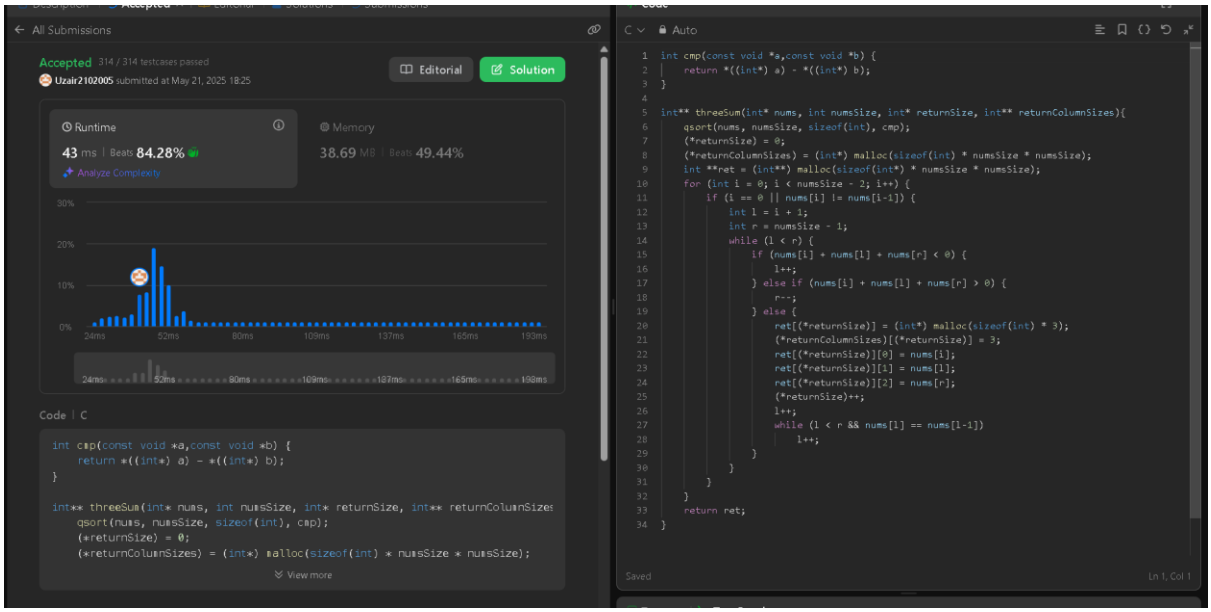
```
    printf("Sorted Array\n");
    for (int i = 0; i < n; ++i) {
        printf("%d ", a[i]);
    }
    printf("\n");


    printf("%.30f\n", t);


    free(a);

    return 0;
}


void mergeSort(int a[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, high, mid);
    }
}


void merge(int a[], int low, int high, int mid) {
    int *tmp = (int*)malloc((high - low + 1) * sizeof(int));
    int i = low, j = mid + 1, k = 0;
```

```
    while (i <= mid && j <= high) {
        if (a[i] < a[j]) {
            tmp[k++] = a[i++];
        } else {
            tmp[k++] = a[j++];
        }
    }

    while (i <= mid) {
        tmp[k++] = a[i++];
    }


    while (j <= high) {
        tmp[k++] = a[j++];
    }


    for (int i = 0; i < (high - low + 1); ++i) {
        a[low + i] = tmp[i];
    }

    free(tmp);
}
```

```
Enter Number of Elements10
Sorted Array
424238335 596516649 719885386 846930886 1189641421 1649760492 1681692777
    1714636915 1804289383 1957747793
0.000003999999999999999818992447


=== Code Execution Successful ===
```

Output

LeetCode 15: 3Sum



Output

Lab program 4:

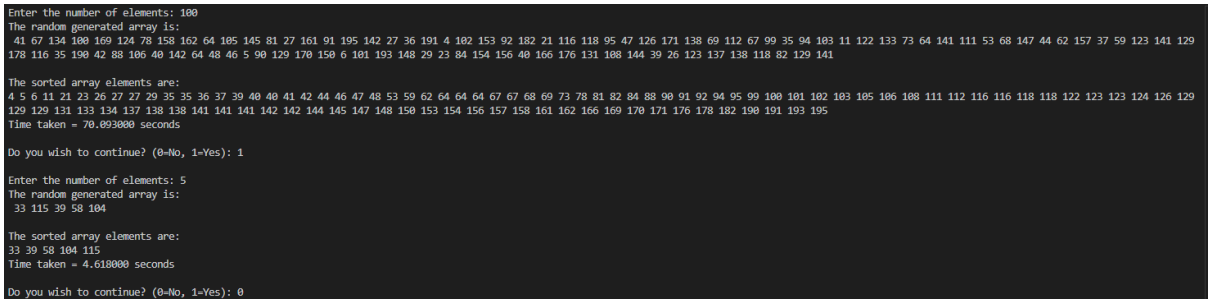
Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <time.h>
#include <stdio.h>
#include <stdlib.h>  // for rand()
#include <windows.h> // for Sleep()

#define MAX 5000

void quicksort(int[], int, int);
int partition(int[], int, int);

int main()
{
    int i, n, a[MAX], ch = 1;  // Initialize ch = 1
    clock_t start, end;
```

```
    while(ch)
    {
        printf("\nEnter the number of elements: ");
        scanf("%d", &n);

        for(i = 0; i < n; i++)
            a[i] = rand() % 200;

        printf("The random generated array is:\n");
        for(i = 0; i < n; i++)
            printf(" %d", a[i]);

        start = clock();
        quicksort(a, 0, n - 1);
        end = clock();

        printf("\n\nThe sorted array elements are:\n");
        for(i = 0; i < n; i++)
            printf("%d ", a[i]);

        printf("\nTime taken = %f seconds", (double)(end - start) / CLOCKS_PER_SEC);
        printf("\n\nDo you wish to continue? (0=No, 1=Yes): ");
        scanf("%d", &ch);
    }
    return 0;
}

void quicksort(int a[], int low, int high)
{
```

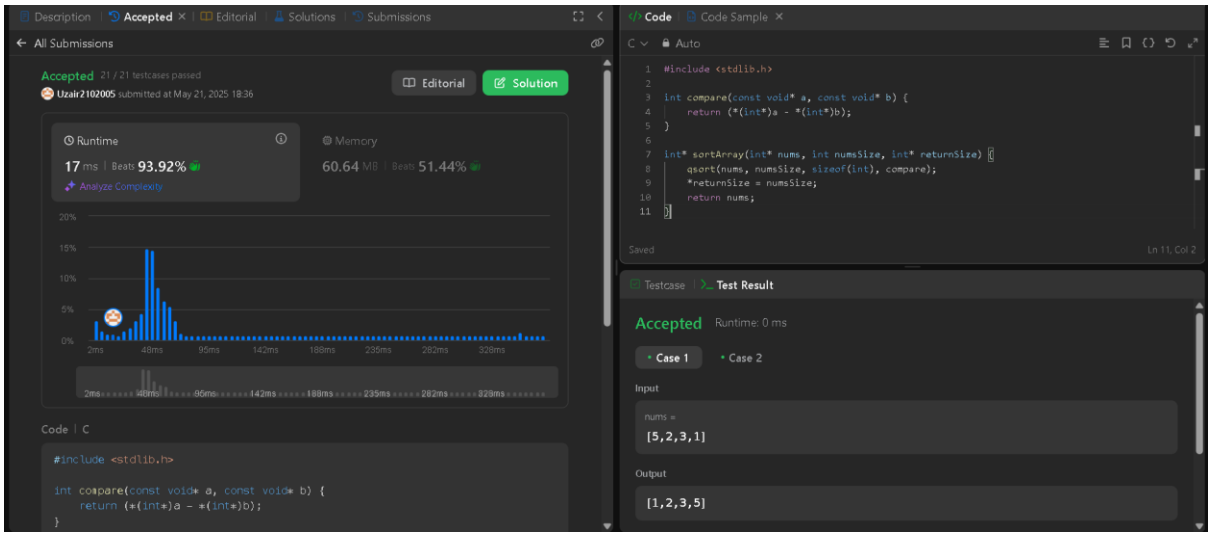
```
    int mid;
    Sleep(500); // Windows-specific delay (500ms)
    if(low < high)
    {
        mid = partition(a, low, high);
        quicksort(a, low, mid - 1);
        quicksort(a, mid + 1, high);
    }
}

int partition(int a[], int low, int high)
{
    int key = a[low];
    int i = low + 1;
    int j = high;

    while(i <= j)
    {
        while(i <= high && key >= a[i])
            i++;
        while(key < a[j])
            j--;

        if(i < j)
        {
            int temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
```

```
        else
        {
            int temp = a[j];
            a[j] = a[low];
            a[low] = temp;
        }
    }
    return j;
}
```



Output

LeetCode 912: Sort an Array



Output

Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>
#include <time.h>

void heapify(int a[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;
    int temp;

    if (left < n && a[left] > a[largest])
        largest = left;

    if (right < n && a[right] > a[largest])
        largest = right;

    if (largest != i) {
        temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;

        heapify(a, n, largest);
    }
}

void heapSort(int a[], int n) {
```

```
    int temp;
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(a, n, i);
    }

    for (int i = n - 1; i >= 0; i--) {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}

int main() {
    int n, a[20], ch = 1;
    clock_t start, end;

    while (ch) {
        printf("\nEnter the number of elements to sort: ");
        scanf("%d", &n);

        printf("Enter the elements to sort: ");
        for (int i = 0; i < n; i++) {
            scanf("%d", &a[i]);
        }
```

```
        start = clock();
        heapSort(a, n);
        end = clock();

        printf("\nThe sorted list of elements is:\n");
        for (int i = 0; i < n; i++) {
            printf("%d\n", a[i]);
        }

        printf("\nTime taken is %lf seconds\n", (double)(end - start) / CLOCKS_PER_SEC);

        printf("Do you wish to run again (0/1)? ");
        scanf("%d", &ch);
    }

    return 0;
}
```

```
Enter the number of elements to sort: 5
Enter the elements to sort:  1 4 2 3 7

The sorted list of elements is:
1
2
3
4
7

Time taken is 0.000000 seconds
Do you wish to run again (0/1)? 0
```

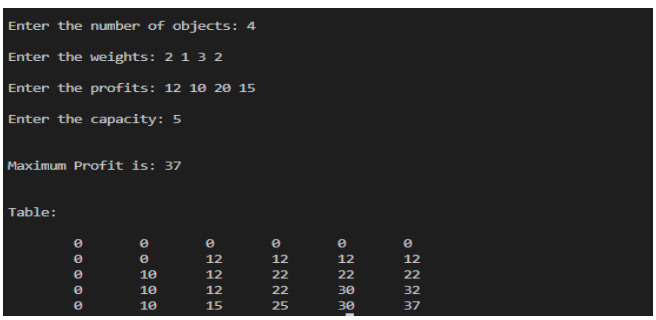
Output

Lab program 6:

Implement 0/1 Knapsack problem using dynamic programming.

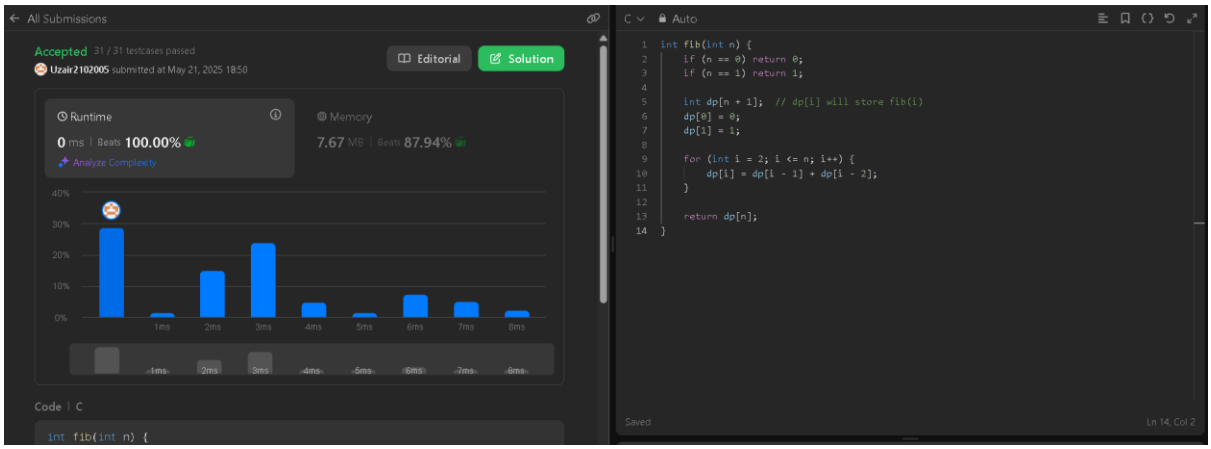
```
#include <stdio.h>

int i, j, n, c, w[10], p[10], v[10][10];


int max(int a, int b) {
   return (a > b) ? a : b;
}
void knapsack(int n, int w[10], int p[10], int c) {
   for (i = 0; i <= n; i++) {
      for (j = 0; j <= c; j++) {
         if (i == 0 || j == 0)
            v[i][j] = 0;
         else if (w[i] > j)
            v[i][j] = v[i - 1][j];
         else
            v[i][j] = max(v[i - 1][j], v[i - 1][j - w[i]] + p[i]);
      }
   }
   printf("\n\nMaximum Profit is: %d\n", v[n][c]);
   printf("\n\nTable:\n\n");
   for (i = 0; i <= n; i++) {
      for (j = 0; j <= c; j++) {
         printf("\t%d", v[i][j]);
      }
      printf("\n");
   }
```

```
}

void main() {
   printf("\nEnter the number of objects: ");
   scanf("%d", &n);

   printf("\nEnter the weights: ");
   for (i = 1; i <= n; i++) {
      scanf("%d", &w[i]);
   }

   printf("\nEnter the profits: ");
   for (i = 1; i <= n; i++) {
      scanf("%d", &p[i]);
   }

   printf("\nEnter the capacity: ");
   scanf("%d", &c);

   knapsack(n, w, p, c);
}
```



Output

LeetCode 509: Fibonacci Number



Output

Lab program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>


int n;
int a[10][10];
int D[10][10];
void write_data() {
    int i, j;
    printf("The Distance matrix is shown below\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            printf("%d", D[i][j]);
            printf(" ");
        }
        printf("\n");
    }
}
```

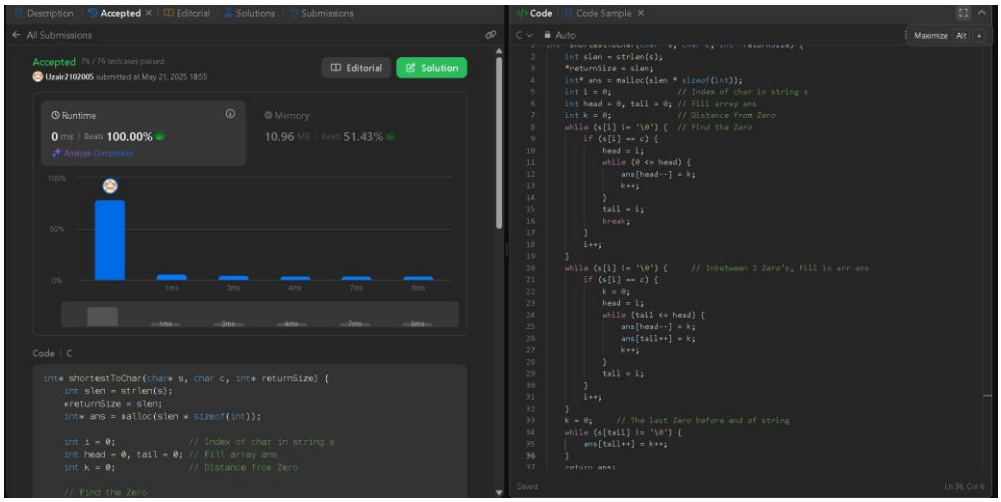
```
void read_data() {
    int i, j;
    printf("Enter the no of nodes\n");
    scanf("%d", &n);
    printf("Enter the adjacency matrix\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }

}
int min(int a, int b) {
    if (a < b) {
        return a;
    } else {
        return b;
    }
}
void Floyd() {
    int i, j, k;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            D[i][j] = a[i][j];
        }
    }
```

```
    for (k = 0; k < n; k++) {
        for (i = 0; i < n; i++) {
            for (j = 0; j < n; j++) {
                D[i][j]=min(D[i][j],(D[i][k]+D[k][j]));
            }
        }
    }
}
int main() {
    read_data();
    Floyd();
    write_data();
    return 0;
}
```

```
Enter the no of nodes
4
Enter the adjacency matrix
0 99 3 99
2 0 99 99
99 6 0 1
7 99 99 0
The Distance matrix is shown below
0 9 3 4
2 0 5 6
8 6 0 1
7 16 10 0
```

Output

LeetCode 821: Shortest Distance to a Character



Output

Lab program 8:

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>

int cost[10][10], n, t[10][2], sum;

void prims(int cost[10][10], int n);

int main() {
    int i, j;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    printf("Enter the cost adjacency matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
```

```
        }
    }

    prims(cost, n);

    printf("Edges of the minimal spanning tree:\n");
    for (i = 0; i < n - 1; i++) {
        printf("(%d, %d) ", t[i][0], t[i][1]);
    }
    printf("\nSum of minimal spanning tree: %d\n", sum);

    return 0;
}

void prims(int cost[10][10], int n) {
    int i, j, u, v;
    int min, source;
    int p[10], d[10], s[10];

    min = 999;
    source = 0;

    // Initialize arrays
    for (i = 0; i < n; i++) {
        d[i] = cost[source][i];
        s[i] = 0;
        p[i] = source;
```

```
}
s[source] = 1;
sum = 0;
int k = 0;

// Find MST
for (i = 0; i < n - 1; i++) {
   min = 999;
   u = -1;

   // Find the vertex with minimum distance to the MST
   for (j = 0; j < n; j++) {
      if (s[j] == 0 && d[j] < min) {
         min = d[j];
         u = j;
      }
   }

   if (u != -1) {
      // Add edge to MST
      t[k][0] = u;
      t[k][1] = p[u];
      k++;
      sum += cost[u][p[u]];
      s[u] = 1;

      // Update distances
```

```
        for (v = 0; v < n; v++) {
            if (s[v] == 0 && cost[u][v] < d[v]) {
                d[v] = cost[u][v];
                p[v] = u;
            }
        }
    }
  }
}
```

```
Enter the number of vertices: 4
Enter the cost adjacency matrix:
0 1 5 2
1 0 99 99
5 99 0 3
2 99 3 0
Edges of the minimal spanning tree:
(1, 0) (3, 0) (2, 3)
Sum of minimal spanning tree: 6
```

Output

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>

int cost[10][10], n, t[10][2], sum;

void kruskal(int cost[10][10], int n);
int find(int parent[10], int i);

int main() {
   int i, j;
```

```
    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    printf("Enter the cost adjacency matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    kruskal(cost, n);

    printf("Edges of the minimal spanning tree:\n");
    for (i = 0; i < n - 1; i++) {
        printf("(%d, %d) ", t[i][0], t[i][1]);
    }
    printf("\nSum of minimal spanning tree: %d\n", sum);

    return 0;
}

void kruskal(int cost[10][10], int n) {
    int min, u, v, count, k;
    int parent[10];

    k = 0;
    sum = 0;
```

```
// Initialize parent array for Union-Find
for (int i = 0; i < n; i++) {
    parent[i] = i;
}

count = 0;
while (count < n - 1) {
    min = 999;
    u = -1;
    v = -1;

    // Find the minimum edge
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (find(parent, i) != find(parent, j) && cost[i][j] < min) {
                min = cost[i][j];
                u = i;
                v = j;
            }
        }
    }

    // Perform Union operation
    int root_u = find(parent, u);
    int root_v = find(parent, v);
```

```
        if (root_u != root_v) {
            parent[root_u] = root_v;
            t[k][0] = u;
            t[k][1] = v;
            sum += min;
            k++;
            count++;
        }
    }
}

int find(int parent[10], int i) {
    while (parent[i] != i) {
        i = parent[i];
    }
    return i;
}
```

```
PS C:\Users\STUDENT\Desktop\KRUSKALS> cd "c:\Users\STUDENT\Desktop\KRUSKALS\" ; if ($?) { gcc kruskals.c -o kruskals } ; if ($?) { .\kruskals }
Enter the number of vertices: 4
Enter the cost adjacency matrix:
0 1 5 2
1 0 99 99
5 99 0 3
2 99 3 0
Edges of the minimal spanning tree:
(0, 1) (0, 3) (2, 3)
Sum of minimal spanning tree: 6
PS C:\Users\STUDENT\Desktop\KRUSKALS>
```

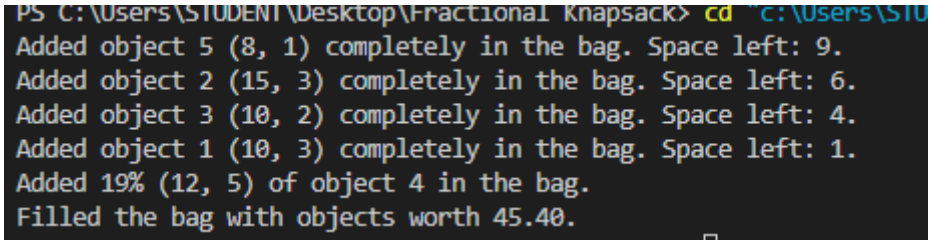
Output

Lab program 9:

Implement Fractional Knapsack using Greedy technique.

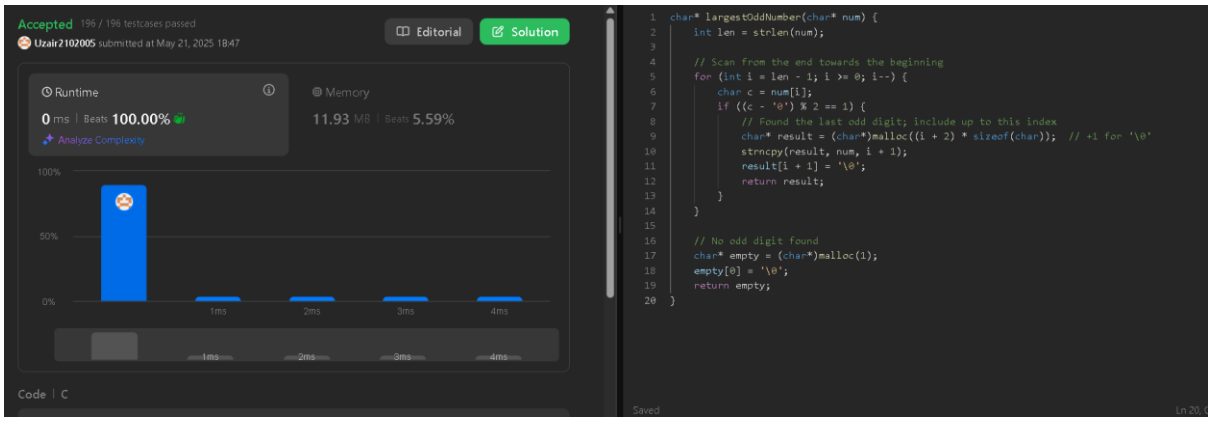
```
#include <stdio.h>


int n = 5;

int p[10] = {3, 3, 2, 5, 1}; /*Weight of items*/

int w[10] = {10, 15, 10, 12, 8};/*Profit of items*/

int W = 10;


int main() {

   int cur_w;

   float tot_v = 0;

   int i, maxi;

   int used[10];


   for (i = 0; i < n; ++i)

      used[i] = 0;


   cur_w = W;


   while (cur_w > 0) {

      maxi = -1;


      for (i = 0; i < n; ++i)

         if ((used[i] == 0) && ((maxi == -1) || ((float)w[i] / p[i] > (float)w[maxi] / p[maxi])))

            maxi = i;


      used[maxi] = 1;
```

```
        cur_w -= p[maxi];
        tot_v += w[maxi];

        if (cur_w >= 0)
            printf("Added object %d (%d, %d) completely in the bag. Space left: %d.\n", maxi + 1, w[maxi], p[maxi], cur_w);
        else {
            printf("Added %d%% (%d, %d) of object %d in the bag.\n", (int)((1 + (float)cur_w / p[maxi]) * 100), w[maxi], p[maxi], maxi + 1);
            tot_v -= w[maxi];
            tot_v += (1 + (float)cur_w / p[maxi]) * w[maxi];
        }
    }

    printf("Filled the bag with objects worth %.2f.\n", tot_v);
    return 0;
}
```



Output

LeetCode 1903: Largest Odd Number in a String



Output

Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>


void main() {
    int i, j, n, v, k, min, u, c[20][20], s[20], d[20];



    printf("\nEnter the number of vertices: ");
    scanf("%d", &n);


    printf("\nEnter the cost adjacency matrix:");
    printf("\n(Enter 999 for no edge)\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            scanf("%d", &c[i][j]);
        }
```

```
}

printf("\nEnter the source vertex: ");
scanf("%d", &v);

// Initialize distances and visited set
for (i = 1; i <= n; i++) {
    s[i] = 0;
    d[i] = c[v][i];
}

d[v] = 0;
s[v] = 1;

for (k = 2; k <= n; k++) {
    min = 999;

    // Find the minimum distance vertex from the set of vertices not yet processed
    for (i = 1; i <= n; i++) {
        if ((s[i] == 0) && (d[i] < min)) {
            min = d[i];
            u = i;
        }
    }

    s[u] = 1;
```

```
        // Update distance value of the adjacent vertices
        for (i = 1; i <= n; i++) {
            if (s[i] == 0) {
                if (d[i] > (d[u] + c[u][i])) {
                    d[i] = d[u] + c[u][i];
                }
            }
        }
    }

    printf("\nThe shortest distances from vertex %d are:\n", v);
    for (i = 1; i <= n; i++) {
        printf("%d --> %d = %d\n", v, i, d[i]);
    }

}
```

```
Enter the number of vertices: 5

Enter the cost adjacency matrix:
(Enter 999 for no edge)
999 7 3 999 999
7 999 2 5 4
3 2 999 4 999
999 5 4 999 6
999 4 999 6 999

Enter the source vertex: 2

The shortest distances from vertex 2 are:
2 --> 1 = 5
2 --> 2 = 0
2 --> 3 = 2
2 --> 4 = 5
2 --> 5 = 4
```

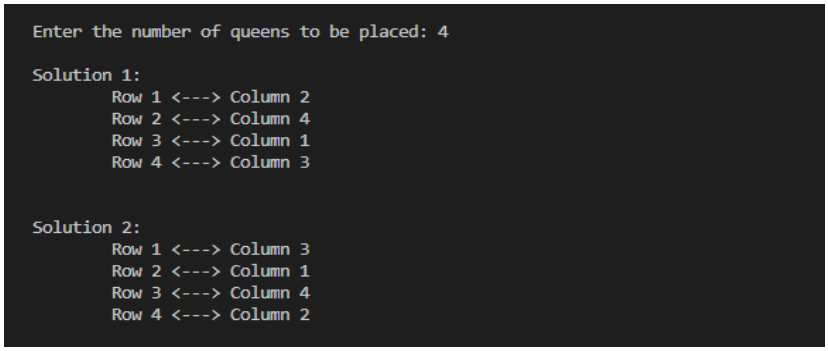
Output

Lab program 11:

Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>
#include <math.h>

int x[20], count = 1;  // Array to store the positions of the queens

void queens(int k, int n);
int place(int k, int j);

int main() {
    int n, k = 1;
    printf("\nEnter the number of queens to be placed: ");
    scanf("%d", &n);
    queens(k, n);
    return 0;
}

void queens(int k, int n) {
    int j;
    for (j = 1; j <= n; j++) {
        if (place(k, j)) {
            x[k] = j;  // Place the queen in the current position
            if (k == n) {  // If all queens are placed
                printf("\nSolution %d:\n", count);
                count++;
                for (int i = 1; i <= n; i++) {
```

```
                printf("\tRow %d <---> Column %d\n", i, x[i]);
            }
            printf("\n");
        } else {
            queens(k + 1, n);  // Try to place the next queen
        }
    }
  }
}

int place(int k, int j) {
  for (int i = 1; i < k; i++) {
    // Check if the queen is in the same column or on the same diagonal
    if (x[i] == j || abs(x[i] - j) == abs(i - k)) {
      return 0;  // Conflict detected
    }
  }
  return 1;  // No conflict, safe to place the queen
}
```



Output